

Q1) Perform the following network commands

1. Ping
2. if config
3. netstat
4. nslookup
5. dig
6. Arp
7. route
8. hostname
9. nmap
10. nmcli
11. Ss
12. mtr
13. ip
14. iwconfig

Credit-Based Flow Control

```
#include <iostream>
#include <thread>
#include <chrono>
using namespace std;
void creditBasedFlowControl(int totalPackets, int initialCredits) {
    int credits = initialCredits;
    for (int packet = 1; packet <= totalPackets; ++packet) {
        if (credits > 0) {
            cout << "Sending packet " << packet << " with " << credits << " credits left." << endl;
            credits--; // Decrease credits as we send packets
            this_thread::sleep_for(chrono::seconds(1));
        } else {
            cout << "No credits left. Waiting for credit replenishment..." << endl;

            this_thread::sleep_for(chrono::seconds(2)); // Simulate replenishment delay
            credits = initialCredits; // Replenish credits
        }
        cout << "Acknowledgment received for packet " << packet << "." << endl;
    }
}
int main() {
    int totalPackets, initialCredits;
    cout << "Enter the total number of packets to send: ";
    cin >> totalPackets;
    cout << "Enter the initial number of credits: ";
    cin >> initialCredits;
    creditBasedFlowControl(totalPackets, initialCredits);
    return 0;
}
```

Sample Input/Output for Credit-Based Flow Control:

Input:

Enter the total number of packets to send: 5
Enter the initial number of credits: 2

Output:

Sending packet 1 with 2 credits left.
Acknowledgment received for packet 1.
Sending packet 2 with 1 credits left.
Acknowledgment received for packet 2.
No credits left. Waiting for credit replenishment...
Sending packet 3 with 2 credits left.
Acknowledgment received for packet 3.
Sending packet 4 with 1 credits left.
Acknowledgment received for packet 4.
No credits left. Waiting for credit replenishment...
Sending packet 5 with 2 credits left.
1. Acknowledgment received for packet 5.

Sliding Window Protocol (Go-Back-N)

```
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <thread>
#include <chrono>
using namespace std;
void goBackN(int totalPackets, int windowSize) {
    int nextPacket = 1;
    while (nextPacket <= totalPackets) {
        for (int i = 0; i < windowSize && nextPacket + i <= totalPackets; ++i) {
            cout << "Sending packet " << nextPacket + i << "..." << endl;
        }
        for (int i = 0; i < windowSize && nextPacket + i <= totalPackets; ++i) {
            int ack = rand() % 2;
            if (ack == 1) {
                this_thread::sleep_for(chrono::seconds(1));
                cout << "Acknowledgment received for packet " << nextPacket + i << "." << endl;
            } else {
                cout << "Acknowledgment lost for packet " << nextPacket + i << ". Retransmitting window..." << endl;
                break;
            }
        }
        nextPacket += windowSize;
    }
}

int main() {
    srand(time(0)); // Initialize random seed
    int totalPackets, windowSize;
    cout << "Enter the total number of packets to send: ";
    cin >> totalPackets;
    cout << "Enter the window size: ";
    cin >> windowSize;
    goBackN(totalPackets, windowSize);
    return 0;
}
```

Sample Input/Output for Go-Back-N:

Input:

Enter the total number of packets to send: 5
Enter the window size: 3

Output:

Sending packet 1...
Sending packet 2...
Sending packet 3...
Acknowledgment received for packet 1.
Acknowledgment lost for packet 2. Retransmitting window...
Sending packet 2...
Sending packet 3...
Sending packet 4...
Acknowledgment received for packet 2.
Acknowledgment received for packet 3.
 Acknowledgment received for packet 4.

```
#include <stdio.h>
// Function to calculate even parity for a byte
int calculateParity(int data[], int size) {
    int parity = 0;
    for (int i = 0; i < size; i++) {
        parity ^= data[i];
    }
    return parity;
}
int main() {
    int data[8];
    int size = 8;
    // Input 8-bit data from the user
    printf("Enter 8-bit data (1 or 0 for each bit):\n");
    for (int i = 0; i < size; i++) {
        printf("Bit %d: ", i + 1);
        scanf("%d", &data[i]);
    }
    // Calculate parity bit (for even parity)
    int parity = calculateParity(data, size);
    printf("Calculated Parity Bit (Even Parity): %d\n", parity);

    // Append the parity bit to the data for transmission
    int transmittedData[9];
    for (int i = 0; i < size; i++) {
        transmittedData[i] = data[i];
    }
    transmittedData[size] = parity;
    // Simulate reception and check parity at the receiver's end
    int receivedParity = calculateParity(transmittedData, size + 1);
    if (receivedParity == 0) {
        printf("No error detected. Data received correctly.\n");
    } else {
        printf("Error detected in the received data.\n");
    }
    return 0;
}
```

Output

Enter 8-bit data (1 or 0 for each bit):

Bit 1: 1

Bit 2: 0

Bit 3: 1
Bit 4: 1
Bit 5: 0
Bit 6: 0
Bit 7: 1
Bit 8: 0
Calculated Parity Bit (Even Parity): 0
No error detected. Data received correctly.

Hamming code

```
#include <stdio.h>
// Function to calculate the parity bits
void calculateHamming(int data[], int hammingCode[]) {
    // Data bits
    hammingCode[2] = data[0];

    hammingCode[4] = data[1];
    hammingCode[5] = data[2];
    hammingCode[6] = data[3];
    // Parity bits
    hammingCode[0] = hammingCode[2] ^ hammingCode[4] ^ hammingCode[6];
    hammingCode[1] = hammingCode[2] ^ hammingCode[5] ^ hammingCode[6];
    hammingCode[3] = hammingCode[4] ^ hammingCode[5] ^ hammingCode[6];
} // Function to check and correct errors
int checkHamming(int hammingCode[]) {
    int c1 = hammingCode[0] ^ hammingCode[2] ^ hammingCode[4] ^ hammingCode[6];
    int c2 = hammingCode[1] ^ hammingCode[2] ^ hammingCode[5] ^ hammingCode[6];
    int c4 = hammingCode[3] ^ hammingCode[4] ^ hammingCode[5] ^ hammingCode[6];
    int errorPos = c1 + (c2 * 2) + (c4 * 4);
    return errorPos;
}
int main() {
    int data[] = {1, 0, 1, 1}; // 4-bit data
    int hammingCode[7];
    // Generate Hamming Code
    calculateHamming(data, hammingCode);
    // Print Hamming Code
    printf("Hamming Code: ");
    for (int i = 0; i < 7; i++) {
        printf("%d", hammingCode[i]);
    }
    printf("\n");
    // Simulate a single-bit error in transmission
    hammingCode[2] = 0; // Intentionally flipping a bit
    // Detect and correct error
    int errorPos = checkHamming(hammingCode);
    if (errorPos == 0) {
        printf("No error detected.\n");
    } else {
        printf("Error detected at position: %d\n", errorPos);
        hammingCode[errorPos - 1] = !hammingCode[errorPos - 1];
        printf("Corrected Hamming Code: ");
        for (int i = 0; i < 7; i++) {
```

```

printf("%d", hammingCode[i]);
}
printf("\n");
}
return 0;
}

```

Output

```

Hamming Code: 0110011
Error detected at position: 3
Corrected Hamming Code: 0110011

```

Stop and wait protocol

```

#include <iostream>
#include <cstdlib>
#include <ctime>
#include <thread>
#include <chrono>
using namespace std;
// Simulates the Stop-and-Wait protocol for flow control
void stopAndWait(int totalPackets) {
    for (int packet = 1; packet <= totalPackets; ++packet) {
        cout << "Sending packet " << packet << "..." << endl;
        // Simulate random acknowledgment loss or delay

        int ack = rand() % 2;
        // If ack = 1, the acknowledgment is received; else, it's lost.
        if (ack == 1) {
            // Simulate transmission delay
            this_thread::sleep_for(chrono::seconds(1));
            cout << "Acknowledgment received for packet " << packet << "." << endl;
        } else {
            // Simulate lost acknowledgment; retransmit the packet
            cout << "Acknowledgment lost for packet " << packet << ". Retransmitting..." << endl;
            packet--; // Retransmit the packet
        }
        // Simulate delay between packet transmissions
        this_thread::sleep_for(chrono::seconds(1));
    }
}

int main() {
    srand(time(0)); // Initialize random seed for acknowledgment loss simulation
    int totalPackets;
    // Input: Total number of packets to send
    cout << "Enter the total number of packets to send: ";
    cin >> totalPackets;
    // Simulate the Stop-and-Wait protocol
    stopAndWait(totalPackets);
    return 0;
}

```

Sample Input/Output:

Enter the total number of packets to send: 5

Possible Output:

Sending packet 1...
Acknowledgment received for packet 1.
Sending packet 2...
Acknowledgment received for packet 2.
Sending packet 3...

Acknowledgment lost for packet 3. Retransmitting...
Sending packet 3...
Acknowledgment received for packet 3.
Sending packet 4...
Acknowledgment received for packet 4.
Sending packet 5...
Acknowledgment lost for packet 5. Retransmitting...
Sending packet 5...
Acknowledgment received for packet 5.

3) TCP AND UDP

Server (UDP Broadcast):

```
#include <stdio.h>
#include <string.h>
#include <arpa/inet.h>
#define PORT 8080
int main() {
    int sockfd;
    struct sockaddr_in broadcast_addr;
    char message[] = "Broadcast message from server";
    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    broadcast_addr.sin_family = AF_INET;
    broadcast_addr.sin_port = htons(PORT);
    broadcast_addr.sin_addr.s_addr = inet_addr("255.255.255.255");
    while (1) {
        sendto(sockfd, message, strlen(message), 0, (struct sockaddr*)&broadcast_addr,
            sizeof(broadcast_addr));
        sleep(1);
    }
    close(sockfd);
    return 0;
}
```

Client (UDP Broadcast Listener):

```
#include <stdio.h>
#include <string.h>
#include <arpa/inet.h>
#define PORT 8080
int main() {
    int sockfd;
    struct sockaddr_in listen_addr;
    char buffer[1024] = {0};
    socklen_t addr_len = sizeof(listen_addr);
    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    listen_addr.sin_family = AF_INET;
    listen_addr.sin_port = htons(PORT);
    listen_addr.sin_addr.s_addr = INADDR_ANY;
```

```

bind(sockfd, (struct sockaddr*)&listen_addr, sizeof(listen_addr));
while (1) {
recvfrom(sockfd, buffer, 1024, 0, (struct sockaddr*)&listen_addr, &addr_len);
printf("Received: %s\n", buffer);
}
close(sockfd);
return 0;
}

```

Output:

- Server continuously broadcasts "Broadcast message from server".
- Clients display this message.

TCP

```

#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#define PORT 8080
int main() {
int server_fd, new_socket;
struct sockaddr_in address;
int addrlen = sizeof(address);
char buffer[1024] = {0};
server_fd = socket(AF_INET, SOCK_STREAM, 0);
address.sin_family = AF_INET;
address.sin_addr.s_addr = INADDR_ANY;
address.sin_port = htons(PORT);
bind(server_fd, (struct sockaddr *)&address, sizeof(address));
listen(server_fd, 3);
new_socket = accept(server_fd, (struct sockaddr *)&address, (socklen_t*)&addrlen);
while (1) {
read(new_socket, buffer, 1024);
printf("Client: %s", buffer);
printf("Enter message: ");
fgets(buffer, 1024, stdin);
send(new_socket, buffer, strlen(buffer), 0);
}
return 0;
}

```

Client (Chat):

```

#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#define PORT 8080
int main() {
int sock;
struct sockaddr_in serv_addr;
char buffer[1024] = {0};
sock = socket(AF_INET, SOCK_STREAM, 0);
serv_addr.sin_family = AF_INET;

serv_addr.sin_port = htons(PORT);

```

```

inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr);
connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr));
while (1) {
printf("Enter message: ");
fgets(buffer, 1024, stdin);
send(sock, buffer, strlen(buffer), 0);
read(sock, buffer, 1024);
printf("Server: %s", buffer);
}
close(sock);
return 0;
}

```

Output:

- Client and server can send and receive messages in a continuous chat.

4)Steps to Simulate RIP in Cisco Packet Tracer 8.1

Step 1: Set Up Network Topology

1. **Open Cisco Packet Tracer version 8.1.**
2. **Create a simple network topology** with three routers connected in a line or triangle configuration. Each router will have one PC connected to it to simulate different subnets.
 - **Drag and drop** 3 routers and 3 PCs from the device list onto the workspace.
3. **Connect the routers** using **serial cables** or **cross-over cables** if using FastEthernet interfaces. Connect the routers to the PCs using **straight-through cables**.

Step 2: Assign IP Addresses to Interfaces

1. **Assign IP addresses to the router interfaces and PCs** to create distinct subnets for each router and the connected PCs.

Here's an example network layout:

- **Router 1 (R1):** Connects to **Router 2 (R2)** via Serial interface.
- **Router 2 (R2):** Connects to both **Router 1 (R1)** and **Router 3 (R3)**.
- **Router 3 (R3):** Connects to **Router 2 (R2)**.

IP Addresses:

- Router 1: FastEthernet 192.168.1.1/24 and Serial 10.0.0.1/30.
- Router 2: Serial 10.0.0.2/30 and 10.0.0.5/30, FastEthernet 192.168.2.1/24.
- Router 3: Serial 10.0.0.6/30 and FastEthernet 192.168.3.1/24.

PC IPs:

- PC 1: 192.168.1.2/24 (Gateway: 192.168.1.1).
- PC 2: 192.168.2.2/24 (Gateway: 192.168.2.1).
- PC 3: 192.168.3.2/24 (Gateway: 192.168.3.1).

Code to Configure IP Addresses on Routers

1. Router 1 configuration:

```
Router1> enable
Router1# configure terminal
Router1(config)# interface fastethernet 0/0
Router1(config-if)# ip address 192.168.1.1 255.255.255.0
Router1(config-if)# no shutdown
Router1(config-if)# exit
Router1(config)# interface serial 0/0/0
Router1(config-if)# ip address 10.0.0.1 255.255.255.252
Router1(config-if)# clock rate 64000
Router1(config-if)# no shutdown
Router1(config-if)# exit
```

2. Router 2 configuration:

```
Router2> enable
Router2# configure terminal
Router2(config)# interface fastethernet 0/0
Router2(config-if)# ip address 192.168.2.1 255.255.255.0
Router2(config-if)# no shutdown
Router2(config-if)# exit
Router2(config)# interface serial 0/0/0
Router2(config-if)# ip address 10.0.0.2 255.255.255.252
Router2(config-if)# no shutdown
Router2(config-if)# exit
Router2(config)# interface serial 0/0/1
Router2(config-if)# ip address 10.0.0.5 255.255.255.252
Router2(config-if)# no shutdown
Router2(config-if)# exit
```

3. Router 3 configuration:

```
Router3> enable
Router3# configure terminal
Router3(config)# interface fastethernet 0/0
Router3(config-if)# ip address 192.168.3.1 255.255.255.0
Router3(config-if)# no shutdown
Router3(config-if)# exit
Router3(config)# interface serial 0/0/0
Router3(config-if)# ip address 10.0.0.6 255.255.255.252
Router3(config-if)# no shutdown
Router3(config-if)# exit
```

Step 3: Configure RIP on All Routers

To enable **RIP v2** on all routers, follow these steps:

1. Router 1 configuration:

```
Router1> enable
Router1# configure terminal
Router1(config)# router rip
Router1(config-router)# version 2
Router1(config-router)# network 192.168.1.0
```

```
Router1(config-router)# network 10.0.0.0
Router1(config-router)# no auto-summary
Router1(config-router)# exit
Router1(config)# exit
```

2. Router 2 configuration:

```
Router2> enable
Router2# configure terminal
Router2(config)# router rip
Router2(config-router)# version 2
Router2(config-router)# network 192.168.2.0
Router2(config-router)# network 10.0.0.0
Router2(config-router)# no auto-summary
Router2(config-router)# exit
Router2(config)# exit
```

3. Router 3 configuration:

```
Router3> enable
Router3# configure terminal
Router3(config)# router rip
Router3(config-router)# version 2
Router3(config-router)# network 192.168.3.0
Router3(config-router)# network 10.0.0.0
Router3(config-router)# no auto-summary
Router3(config-router)# exit
Router3(config)# exit
```

Step 4: Verify RIP Configuration

1. Use the following command on each router to **verify the routing table** and ensure RIP has learned the routes from neighboring routers.

```
Router1# show ip route
```

Expected Output on Router 1:

```
Gateway of last resort is not set
R    192.168.2.0/24 [120/1] via 10.0.0.2, 00:00:15, Serial0/0/0
R    192.168.3.0/24 [120/1] via 10.0.0.2, 00:00:15, Serial0/0/0
C    192.168.1.0/24 is directly connected, FastEthernet0/0
C    10.0.0.0/30 is directly connected, Serial0/0/0
```

Expected Output on Router 2:

```
Gateway of last resort is not set
R    192.168.3.0/24 [120/1] via 10.0.0.6, 00:00:15, Serial0/0/1
C    192.168.2.0/24 is directly connected, FastEthernet0/0
C    10.0.0.4/30 is directly connected, Serial0/0/1
C    10.0.0.0/30 is directly connected, Serial0/0/0
R    192.168.1.0/24 [120/1] via 10.0.0.1, 00:00:15, Serial0/0/0
```

Expected Output on Router 3:

```
Gateway of last resort is not set
```

```
R    192.168.1.0/24 [120/1] via 10.0.0.5, 00:00:15, Serial0/0/0
R    192.168.2.0/24 [120/1] via 10.0.0.5, 00:00:15, Serial0/0/0
C    192.168.3.0/24 is directly connected, FastEthernet0/0
C    10.0.0.4/30 is directly connected, Serial0/0/0
```

Step 5: Test Network Connectivity

Once the RIP configuration is complete, you can **ping between devices** in different subnets to check if routing is working correctly.

1. From **PC1** (connected to Router 1) to **PC3** (connected to Router 3):

```
PC1> ping 192.168.3.2
```

Expected Ping Output:

```
Pinging 192.168.3.2 with 32 bytes of data:
Reply from 192.168.3.2: bytes=32 time=12ms TTL=255
Reply from 192.168.3.2: bytes=32 time=10ms TTL=255
Reply from 192.168.3.2: bytes=32 time=11ms TTL=255
```

If the ping is successful, it confirms that RIP has correctly propagated the routes across the network.

Step 6: Save the Configuration

Make sure to save the configuration on each router to avoid losing it after a reboot:

```
Router1# copy running-config startup-config
Router2# copy running-config startup-config
Router3# copy running-config startup-config
```

Conclusion

With these steps and configurations, you can successfully simulate **RIP v2** on Cisco Packet Tracer version 8.1. You'll be able to observe how routing updates are propagated across routers and verify connectivity between devices in different subnets. If you'd like to extend this to OSPF or EIGRP, the configurations would be similar, with protocol-specific commands.

For Cisco Packet Tracer version 8.1, configuring a unicast routing protocol

Updated Steps for Cisco Packet Tracer 8.1

Step 1: Create the Topology

1. **Drag and drop devices:** Add the necessary devices (e.g., routers, switches, and end devices like PCs) from the device list.
 - Select routers (e.g., 3 routers).
 - Add PCs connected to each router's network.
2. **Connect the devices:**

- Use **Straight-through cables** to connect PCs to routers' **FastEthernet** interfaces.
- Use **Cross-over cables** or **Serial cables** to connect routers together using their respective **Serial or FastEthernet interfaces**.

Step 2: IP Address Configuration

1. **Configure the routers with appropriate IP addresses** on the interfaces connecting to other routers and to the PCs.

Code for Router 1

```
Router1> enable
Router1# configure terminal
Router1(config)# interface fastethernet 0/0
Router1(config-if)# ip address 192.168.1.1 255.255.255.0
Router1(config-if)# no shutdown
Router1(config-if)# exit
Router1(config)# interface serial 0/0/0
Router1(config-if)# ip address 10.0.0.1 255.255.255.252
Router1(config-if)# clock rate 64000 (for DCE connection if applicable)
Router1(config-if)# no shutdown
Router1(config-if)# exit
Router1(config)# exit
```

Code for Router 2

```
Router2> enable
Router2# configure terminal
Router2(config)# interface fastethernet 0/0
Router2(config-if)# ip address 192.168.2.1 255.255.255.0
Router2(config-if)# no shutdown
Router2(config-if)# exit
Router2(config)# interface serial 0/0/0
Router2(config-if)# ip address 10.0.0.2 255.255.255.252
Router2(config-if)# no shutdown
Router2(config-if)# exit
Router2(config)# exit
```

Code for Router 3 (if applicable)

```
Router3> enable
Router3# configure terminal
Router3(config)# interface fastethernet 0/0
Router3(config-if)# ip address 192.168.3.1 255.255.255.0
Router3(config-if)# no shutdown
Router3(config-if)# exit
Router3(config)# interface serial 0/0/0
Router3(config-if)# ip address 10.0.0.6 255.255.255.252
Router3(config-if)# no shutdown
Router3(config-if)# exit
Router3(config)# exit
```

Step 3: Configure RIP on All Routers

1. **Enable RIP routing protocol** on all routers. Below are the RIP configurations for each router. RIP Version 2 is used here.

Code for Router 1

```
Router1> enable
Router1# configure terminal
Router1(config)# router rip
Router1(config-router)# version 2
Router1(config-router)# network 192.168.1.0
Router1(config-router)# network 10.0.0.0
Router1(config-router)# no auto-summary
Router1(config-router)# exit
Router1(config)# exit
```

Code for Router 2

```
Router2> enable
Router2# configure terminal
Router2(config)# router rip
Router2(config-router)# version 2
Router2(config-router)# network 192.168.2.0
Router2(config-router)# network 10.0.0.0
Router2(config-router)# no auto-summary
Router2(config-router)# exit
Router2(config)# exit
```

Code for Router 3 (if applicable)

```
Router3> enable
Router3# configure terminal
Router3(config)# router rip
Router3(config-router)# version 2
Router3(config-router)# network 192.168.3.0
Router3(config-router)# network 10.0.0.0
Router3(config-router)# no auto-summary
Router3(config-router)# exit
Router3(config)# exit
```

Step 4: Verify RIP Routing Protocol

To verify that RIP is functioning properly and that the routes have been propagated to other routers, use the following command:

Command to Verify Routes on Each Router

```
Router1# show ip route
```

Expected Output:

```
Gateway of last resort is not set
  192.168.2.0 [120/1] via 10.0.0.2, 00:00:10, Serial0/0/0
  192.168.3.0 [120/1] via 10.0.0.2, 00:00:10, Serial0/0/0
C    192.168.1.0 is directly connected, FastEthernet0/0
C    10.0.0.0/30 is directly connected, Serial0/0/0
```

Step 5: Test Connectivity

You can now **ping** between devices to check if the routing protocol is working as expected.

Command to Ping from Router 1 to PC in Router 2's network:

```
Router1# ping 192.168.2.2
```

Expected Output:

```
Pinging 192.168.2.2 with 32 bytes of data:
Reply from 192.168.2.2: bytes=32 time=20ms TTL=255
Reply from 192.168.2.2: bytes=32 time=15ms TTL=255
Reply from 192.168.2.2: bytes=32 time=12ms TTL=255
```

Step 6: Save Configuration

Save your router configuration so that it persists after reboot:

Command to Save Configuration:

```
Router1# copy running-config startup-config
Router2# copy running-config startup-config
Router3# copy running-config startup-config
```

Expected Output:

```
Destination filename [startup-config]?
Building configuration...
[OK]
```

Step 7: Enable Simulation Mode (Optional)

Switch to **Simulation Mode** in Packet Tracer to visualize the routing process and the packet exchanges. You will see the RIP update packets as well as ICMP (ping) requests.

Conclusion

By following these updated steps and codes, you can successfully simulate the unicast routing protocol (RIP) in Cisco Packet Tracer version 8.1. If you wish to configure different routing protocols such as **OSPF** or **EIGRP**, the process is similar, but the commands will differ slightly.